PIVOT-IV ARCHITECTURE

Instruction Set & Register Architecture Reference Manual

A High-Density Protocol Engine and Embedded Controller Microarchitecture
Standard Load/Store & Dedicated Hardware Register Specification (RISC-Style)

Document Version:	1.0 (Production ISA)
Architecture Class:	Load/Store Accumulator + Register Array
Data Path Width:	8-Bit Architecture (4-Bit Core Nibbles)
Instruction Set:	16 Base Instructions (Fixed 4-Bit Opcode)
Address Space:	16 Memory-Mapped Registers (0x0 – 0xF)
Pipeline:	1-Cycle Execution (Split-Stream Fetch)
Verification Status:	100% Verified in Perl/Selenium Test Suite

1. Architectural Overview

The **PIVOT-IV** is an ultra-compact, high-density accumulator/register RISC-class processor architecture optimized for high-speed protocol emulation (UART, SPI, I2C, GPIO bit-banging) and real-time state machine execution.

1.1 Microarchitectural Highlights

- **1-Cycle Split-Stream Execution:** Instruction memory is physically segregated into *Opcode RAM*, *Operand-1 RAM*, *Operand-2 RAM*, and a *Jump Table ROM*, enabling single-cycle fetch-decode-execute without pipeline stalls or branch delay slots.
- **8-Bit Data Width:** Registers and ALU internal paths process 8-bit payloads directly, perfectly matching standard industrial protocol frames.
- **Integrated Peripheral Subsystem:** Dedicated hardware GPIO pin status, active direction control, output latches, and hardware countdown stall timers are exposed directly in register space.
- **Zero-Overhead Branching:** Jump instructions use a 16-entry `JUMP_TABLE` containing absolute target addresses for single-cycle conditional and unconditional jumps.

1.2 Instruction Formats

Every instruction is identified by a 4-bit Opcode (`[3:0]`). The instruction lengths vary predictably:

Format R (1 Nibble / 4 bits) — Zero-Operand ALU & System
Used by ALU math instructions (`ADD` , `SUB` , `AND` , `XOR` , `SHR` , `SHL` , `NOP`).

3 : 0
OPCODE [3:0]

Format M (2 Nibbles / 8 bits) — Memory, Pin & Branch
Used by `SET0` , `SET1` , `LOAD` , `STORE` , `JMP` , `JMPNZ` , `JMPC` .

7 : 4	3 : 0
OPCODE [3:0]	ADDR / PIN / TAB_ID [3:0]

Format I (3 Nibbles / 12 bits) — 8-Bit Immediate Load
Used by `LOADI imm8` (loads ACC) and `LOADIB imm8` (loads register B).

11 : 8	7 : 4	3 : 0
OPCODE [3:0]	IMM8 [7:4] (High Nibble)	IMM8 [3:0] (Low Nibble)

2. Register Architecture (16-Address Map)

The PIVOT-IV architecture maps both primary compute registers and dedicated hardware peripherals into a unified 16-address memory space (`0x0` to `0xF`).

Addr	Name	Alias	Access	Microarchitectural Purpose & Behavior
`0x0`	**ACC**	`R0`	Read / Write	**Math Accumulator:** Primary working register for all ALU operations, memory load destinations, and store sources.
`0x1`	**B**	`R1`	Read / Write	**Secondary Math Register:** Holds the right-hand operand for two-input ALU instructions (`ACC = ACC op B`).
`0x2`	**R2**	`AUX`	Read / Write	**General Purpose / Loop Counter:** Scratchpad register for state tracking and iteration loops.
`0x3`	**R3**	`TMP`	Read / Write	**General Purpose Scratchpad:** Intermediate calculation storage.
`0x4`	**TX_FIFO**	`R4`	Write (Host)	**Transmit FIFO:** Moving data here streams it to the SPI/UART host interface. *Stalls CPU execution if FIFO is full.*
`0x5`	**RX_FIFO**	`R5`	Read (Host)	**Receive FIFO:** Reading pops byte from the host interface. *Stalls CPU execution if FIFO is empty.*
`0x6`	**PIN_STATE**	`PIN_IN`	Read-Only	**Physical GPIO Status:** Live voltage status of the 4 physical GPIO pins. Direct manual toggle is blocked in hardware.
`0x7`	**TIMER_L**	`TIMER`	Read / Write	**Hardware Countdown Timer:** Writing sets countdown cycles. **Reading stalls CPU until timer reaches zero**, enabling cycle-accurate delays.
`0x8`	**FLAGS**	`STATUS`	Read / Write	**ALU Status & Inter-Core Flags:** Bit 0: Zero Flag (ZF); Bit 1: Carry Flag (CF); Bits 2-5: Valid flags for shared registers.
`0x9`	**SHARED_0**	`SH0`	Read / Write	**Inter-Core Mailbox 0:** Shared across multi-core CPUs. *Stalls CPU on read if data valid bit is 0.*
`0xA`	**SHARED_1**	`SH1`	Read / Write	**Inter-Core Mailbox 1:** Multi-core synchronized mailbox register.
`0xB`	**SHARED_2**	`SH2`	Read / Write	**Inter-Core Mailbox 2:** Multi-core synchronized mailbox register.
`0xC`	**SHARED_3**	`SH3`	Read / Write	**Inter-Core Mailbox 3:** Multi-core synchronized mailbox register.
`0xD`	**PIN_OUT**	`R6`	Read / Write	**GPIO Output Latch:** Holds data written to pins. Active output driven to pins where corresponding `PIN_EN` bit is 1.
`0xE`	**R7**	`SCRATCH`	Read / Write	**General Purpose Register:** Additional scratchpad byte.
`0xF`	**PIN_DIR**	`PIN_EN`	Read / Write	**GPIO Direction / Enable:** 4-bit mask (1 = Active Driven Output, 0 = Input / High-Z). Read returns `{pin_state, pin_dir}` .

3. Complete Instruction Set Reference (All 16 Instructions)

ADD — Add B to Accumulator

ALU Arithmetic

Syntax: `ADD` | **Opcode:** `0000` (0x0) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: `ACC ← (ACC + B) mod 256`

Flags: `CF` set if sum exceeds 255 (or 15 in 4-bit core mode); `ZF` set if result is zero.

SUB — Subtract B from Accumulator

ALU Arithmetic

Syntax: `SUB` | **Opcode:** `0001` (0x1) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: `ACC ← (ACC - B) mod 256`

Flags: `CF` set if no borrow occurred (`ACC >= B`); `ZF` set if `ACC == B` .

AND — Bitwise Logical AND

ALU Logical

Syntax: `AND` | **Opcode:** `0010` (0x2) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: `ACC ← ACC & B`

Flags: `CF ← 0` ; `ZF` set if result is zero.

XOR — Bitwise Logical Exclusive-OR

ALU Logical

Syntax: `XOR` | **Opcode:** `0011` (0x3) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: `ACC ← ACC ^ B`

Flags: `CF ← 0` ; `ZF` set if result is zero.

SHR — Logical Shift Right by 1

ALU Shift

Syntax: `SHR` | **Opcode:** `0100` (0x4) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: `ACC ← ACC >> 1` (0 shifted into MSB, bit 0 shifted into CF).

Flags: `CF ← ACC[0]` ; `ZF` set if result is zero.

SHL — Logical Shift Left by 1

ALU Shift

Syntax: `SHL` | **Opcode:** `0101` (0x5) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: `ACC ← (ACC << 1) mod 256` (0 shifted into LSB, bit 7/3 into CF).

Flags: `CF ← old MSB` ; `ZF` set if result is zero.

NOP — No Operation

Control

Syntax: `NOP` | **Opcode:** `0110` (0x6) | **Format:** Format R (1 Nibble) | **Cycles:** 1

Operation: Consumes 1 clock cycle without state modification. Advances Program Counter.

LOADIB — Load Immediate into Register B

Data Movement

Syntax: `LOADIB #imm8` | **Opcode:** `0111` (0x7) | **Format:** Format I (3 Nibbles) | **Cycles:** 1

Operation: `B ← imm8` (Loads full 8-bit constant directly into secondary math register B).

SET0 — Drive Output Pin Low (0)

GPIO Pin Control

Syntax: `SET0 pin` | **Opcode:** `1000` (0x8) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: Sets `PIN_DIR[pin] = 1` and drives `PIN[pin] = 0` on active clock edge.

SET1 — Drive Output Pin High (1)

GPIO Pin Control

Syntax: `SET1 pin` | **Opcode:** `1001` (0x9) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: Sets `PIN_DIR[pin] = 1` and drives `PIN[pin] = 1` on active clock edge.

LOADI — Load Immediate into Accumulator

Data Movement

Syntax: `LOADI #imm8` | **Opcode:** `1010` (0xA) | **Format:** Format I (3 Nibbles) | **Cycles:** 1

Operation: `ACC ← imm8` (Loads full 8-bit constant directly into primary accumulator ACC).

LOAD — Load Memory Register into Accumulator

Memory Access

Syntax: `LOAD addr` | **Opcode:** `1011` (0xB) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: `ACC ← REG_FILE[addr]` (Loads memory-mapped register at 4-bit address into ACC).

Special Behavior: Reading address `0x7` (TIMER_L) stalls next instruction execution until countdown is 0.

STORE — Store Accumulator into Memory Register

Memory Access

Syntax: `STORE addr` | **Opcode:** `1100` (0xC) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: `REG_FILE[addr] ← ACC` (Stores contents of ACC into memory-mapped register `addr`).

Special Behavior: Storing into address `0xD` (PIN_OUT) drives output pins where `PIN_EN == 1`.

JMP — Unconditional Jump via Table

Branch

Syntax: `JMP id` | **Opcode:** `1101` (0xD) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: `PC ← JUMP_TABLE[id]` (Loads absolute target address from Jump Table entry `id`).

JMPNZ — Jump if Not Zero (ZF == 0)

Branch

Syntax: `JMPNZ id` | **Opcode:** `1110` (0xE) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: If `ZF == 0`, `PC ← JUMP_TABLE[id]`; else advance to next sequential instruction.

JMPC — Jump if Carry (CF == 1)

Branch

Syntax: `JMPC id` | **Opcode:** `1111` (0xF) | **Format:** Format M (2 Nibbles) | **Cycles:** 1

Operation: If `CF == 1`, `PC ← JUMP_TABLE[id]`; else advance to next sequential instruction.

4. Verification & Testing Reference

Every opcode, binary encoding, memory address, and execution stall behavior has been exhaustively validated against the online simulator (`basic_cpu_gui.html`) using Perl and automated headless Chrome/Selenium test suites:

Test Suite	File Path	Tests	Status
ISA Compliance Suite	`tests/test_isa_compliance.pl`	16 opcodes, ALU arithmetic, SET0/1, LOAD/STORE, JMP table	✅ 100% Passed (5/5 subtests)
Timer & Registers Suite	`tests/test_timer_and_registers.pl`	R0..R7 mapping, Timer countdown stall cycles on read	✅ 100% Passed (3/3 subtests)
Validation Suite	`tests/test_validation_import.pl`	Exhaustive syntax, opcode checking, operand bounds, overflow	✅ 100% Passed (5/5 subtests)
GPIO & Waveform Suite	`tests/test_cpu_waveform.pl`	Pin bubble protection, OE drive, future waveform timeline	✅ 100% Passed (5/5 subtests)
Instant Execution Suite	`tests/test_instant_execution.pl`	Live interactive instruction bar execution, flags, and trace	✅ 100% Passed (4/4 subtests)